CM0859 – MT5009 Type Theory
Typed Lambda Calculus (or Proofs as Programs)

Andrés Sicard-Ramírez

Universidad EAFIT

Semester 2025-2

Preliminaries

Textbook

"Lecture Notes on Proofs as Programs" (Pfenning 2023).

Conventions

The numbers and page numbers assigned to chapters, examples, exercises, figures, quotes, sections and theorems on these slides correspond to the numbers assigned in the textbook.

Source code

The AGDA code have been tested with AGDA 2.8.0.

Typed Lambda Calculus (or Proofs as Programs)

Introduction

- Propositions (types): $A, B, C, \ldots$

Typed Lambda Calculus (or Proofs as Programs)

Introduction

- Propositions (types): $A, B, C, \ldots$
- Lambda terms (programs): $M, N, \ldots$

Typed Lambda Calculus (or Proofs as Programs)

Introduction

- Propositions (types): $A, B, C, \ldots$
- Lambda terms (programs): $M, N, \ldots$
- Judgements:
  - (i) $A\,\mathrm{type}$ (A is a type)
  - (ii) $M : A$ (λ-term M is a proof-term for proposition A)
    (program M has type A)
  - (iii) $M \Rightarrow_R M'$ (λ-term M reduces to λ-term M')

(continued on next slide)

Typed Lambda Calculus (or Proofs as Programs)

Introduction (continuation)

- Proposition-as-types principle

  > *"We intend that if* $M : A$ *then* $A\,\mathrm{true}$*. Conversely, if* $A\,\mathrm{true}$ *then* $M : A$ *for some appropriate proof-term* M*. But we want something more: every deduction of* $M : A$ *should correspond to a deduction of* $A\,\mathrm{true}$ *with an identical structure and vice versa." (p. L4.1)*

Typed Lambda Calculus (or Proofs as Programs)

Introduction (continuation)

- Proposition-as-types principle

  "We intend that if $M : A$ then $A\,\mathrm{true}$. Conversely, if $A\,\mathrm{true}$ then $M : A$ for some appropriate proof-term M. But we want something more: every deduction of $M : A$ should correspond to a deduction of $A\,\mathrm{true}$ with an identical structure and vice versa." (p. L4.1)

  *"The introduction rules for a logical connective correspond to the **constructors** for the corresponding type. Conversely, the elimination rules correspond to **destructors**." (p. L4.2)*

Typed Lambda Calculus (or Proofs as Programs)

Introduction (continuation)

- Proposition-as-types principle

  "We intend that if $M : A$ then $A\,\mathrm{true}$. Conversely, if $A\,\mathrm{true}$ then $M : A$ for some appropriate proof-term M. But we want something more: every deduction of $M : A$ should correspond to a deduction of $A\,\mathrm{true}$ with an identical structure and vice versa." (p. L4.1)

  *"The introduction rules for a logical connective correspond to the **constructors** for the corresponding type. Conversely, the elimination rules correspond to **destructors**." (p. L4.2)*

- Form of the inference rules (same than for natural deduction):

$$\frac{J_1 \quad \ldots \quad J_n}{J}\ \text{rule name}$$

  where J (conclusion) and $J_1, \ldots, J_n$ (premises) are all judgements.

(continued on next slide)

Typed Lambda Calculus (or Proofs as Programs)

Introduction (continuation)

- Inference rules (same than for natural deduction): Formation, introduction and elimination rules.

Typed Lambda Calculus (or Proofs as Programs)

Introduction (continuation)

- Inference rules (same than for natural deduction): Formation, introduction and elimination rules.
- New inference rules: Reduction rules.

> *"If proofs are programs then we need to explain how proofs are to be executed, and which results may be returned by a computation."*
>
> *"Reduction arises when a destructor is applied to a constructor."*
>
> *"A proof reduction arises when we apply an elimination rule to the result of an introduction rules." (p. L4.7)*

Typed Lambda Calculus (or Proofs as Programs)

Introduction (continuation)

- Inference rules (same than for natural deduction): Formation, introduction and elimination rules.
- New inference rules: Reduction rules.[†]

> *"If proofs are programs then we need to explain how proofs are to be executed, and which results may be returned by a computation."*
>
> *"Reduction arises when a destructor is applied to a constructor."*
>
> *"A proof reduction arises when we apply an elimination rule to the result of an introduction rules." (p. L4.7)*

[†] Some authors introduce *computation* rules instead of *reduction* rules.

Typed Lambda Calculus (or Proofs as Programs)

Observation

Other names for the proposition-as-types principle are (or should be):

- the Curry-Howard correspondence,
- the Curry-Howard isomorphism (textbook),
- the Brouwer-Heyting-Kolmogorov-Schönfinkel-Curry-Meredith-Kleene-Feys-Gödel-Läuchli-Kreisel-Tait-Lawvere-Howard-de Bruijn-Scott-Martin Löf-Girard-Reynolds-Stenlund-Constable-Coquand-Huet$\cdots$ correspondence
  (Sørensen and Urzyczyn 2006, p. viii).

Typed Lambda Calculus (or Proofs as Programs)

Definition

The **types** of typed lambda calculus are defined by the following grammar:

$$
\begin{array}{lll}
\sigma, \tau ::= \mathbf{b} & & \text{(base type)} \\
\quad \mid \sigma \to \tau & & \text{(function type)} \\
\quad \mid \sigma \times \tau & & \text{(product type)} \\
\quad \mid \sigma + \tau & & \text{((disjoint) sum type or coproduct type)} \\
\quad \mid \mathbf{1} & & \text{(unit type)} \\
\quad \mid \mathbf{0} & & \text{(empty type)}
\end{array}
$$

Typed Lambda Calculus (or Proofs as Programs)

Definition

The λ-**terms** of our typed lambda calculus are defined by the following grammar:

$$
\begin{array}{lll}
M, N, P ::= x & & \text{(variable)} \\
\quad | \ \lambda x.M & & (\lambda\text{-abstraction}) \\
\quad | \ M\ N & & \text{(application)} \\
\quad | \ \langle M\ ,\ N \rangle \mid \mathsf{fst}\ M \mid \mathsf{snd}\ M & & \text{(pair and projections)} \\
\quad | \ \mathsf{inl}\ M \mid \mathsf{inr}\ M \mid \mathsf{case}(M, x.N, y.P) & & \text{(injections and case analysis)} \\
\quad | \ \langle\rangle & & \text{(unit)} \\
\quad | \ \mathsf{abort}\ M & & \text{(error)}
\end{array}
$$

Typed Lambda Calculus (or Proofs as Programs)

Inference rules for conjunction (product type)

Typed Lambda Calculus (or Proofs as Programs)

Inference rules for conjunction (product type)

- Formation rule

$$\frac{A\,\mathrm{prop} \qquad B\,\mathrm{prop}}{A \wedge B\,\mathrm{prop},} \qquad \frac{A\,\mathrm{type} \qquad B\,\mathrm{type}}{A \wedge B\,\mathrm{type}.}$$

(continued on next slide)

Typed Lambda Calculus (or Proofs as Programs)

Inference rules for conjunction (product type) (continuation)

- Introduction rule

$$\frac{A\,\text{true} \qquad B\,\text{true}}{A \wedge B\,\text{true},}\,\wedge\text{I} \qquad \frac{M : A \qquad N : B}{\langle\, M\,,\, N\,\rangle : A \wedge B.}\,\wedge\text{I}$$

Typed Lambda Calculus (or Proofs as Programs)

Inference rules for conjunction (product type) (continuation)

- Introduction rule

$$\frac{A\,\text{true} \qquad B\,\text{true}}{A \wedge B\,\text{true},}\,\wedge\text{I} \qquad \frac{M : A \qquad N : B}{\langle\, M\,,\, N\,\rangle : A \wedge B.}\,\wedge\text{I}$$

- Elimination rules

$$\frac{A \wedge B\,\text{true}}{A\,\text{true},}\,\wedge\text{E}_1 \qquad \frac{M : A \wedge B}{\mathsf{fst}\, M : A,}\,\wedge\text{E}_1$$

$$\frac{A \wedge B\,\text{true}}{B\,\text{true},}\,\wedge\text{E}_2 \qquad \frac{M : A \wedge B}{\mathsf{snd}\, M : B.}\,\wedge\text{E}_2$$

Typed Lambda Calculus (or Proofs as Programs)

Inference rules for conjunction (product type) (continuation)

- Introduction rule

$$\frac{A\,\mathrm{true} \qquad B\,\mathrm{true}}{A \wedge B\,\mathrm{true},}\,\wedge\mathrm{I} \qquad \frac{M : A \qquad N : B}{\langle\, M\,,\, N\,\rangle : A \wedge B.}\,\wedge\mathrm{I}$$

- Elimination rules

$$\frac{A \wedge B\,\mathrm{true}}{A\,\mathrm{true},}\,\wedge\mathrm{E}_1 \qquad \frac{M : A \wedge B}{\mathsf{fst}\, M : A,}\,\wedge\mathrm{E}_1$$

$$\frac{A \wedge B\,\mathrm{true}}{B\,\mathrm{true},}\,\wedge\mathrm{E}_2 \qquad \frac{M : A \wedge B}{\mathsf{snd}\, M : B.}\,\wedge\mathrm{E}_2$$

- Reduction rules

$$\mathsf{fst}\,\langle\, M\,,\, N\,\rangle \Rightarrow_R M,$$
$$\mathsf{snd}\,\langle\, M\,,\, N\,\rangle \Rightarrow_R N.$$

Typed Lambda Calculus (or Proofs as Programs)

Inference rules for implication (function type)

Typed Lambda Calculus (or Proofs as Programs)

Inference rules for implication (function type)

- Formation rule

$$\frac{A\,\mathrm{prop} \qquad B\,\mathrm{prop}}{A \supset B\,\mathrm{prop},} \qquad \frac{A\,\mathrm{type} \qquad B\,\mathrm{type}}{A \supset B\,\mathrm{type}.}$$

(continued on next slide)

Typed Lambda Calculus (or Proofs as Programs)

Inference rules for implication (function type) (continuation)

- Introduction rule

$$\frac{\begin{array}{c}\overline{A\,\mathrm{true}}^{\,u}\\ \vdots\\ B\,\mathrm{true}\end{array}}{A \supset B\,\mathrm{true},}\supset\mathrm{I}^{u} \qquad \frac{\begin{array}{c}\overline{u : A}^{\,u}\\ \vdots\\ M : B\end{array}}{\lambda u.M : A \supset B.}\supset\mathrm{I}^{u}$$

Typed Lambda Calculus (or Proofs as Programs)

Inference rules for implication (function type) (continuation)

- Introduction rule

$$\frac{\begin{array}{c}\overline{A\,\text{true}}^{\,u}\\ \vdots\\ B\,\text{true}\end{array}}{A \supset B\,\text{true},}\supset\text{I}^{u} \qquad \frac{\begin{array}{c}\overline{u : A}^{\,u}\\ \vdots\\ M : B\end{array}}{\lambda u.M : A \supset B.}\supset\text{I}^{u}$$

- Elimination rule

$$\frac{A \supset B\,\text{true} \qquad A\,\text{true}}{B\,\text{true},}\supset\text{E} \qquad \frac{M : A \supset B \qquad N : A}{M\,N : B.}\supset\text{E}$$

Typed Lambda Calculus (or Proofs as Programs)

Inference rules for implication (function type) (continuation)

- Introduction rule

$$\frac{\begin{array}{c}\\ A\,\mathrm{true}\end{array}^{\,u}}{\begin{array}{c}\vdots\\ B\,\mathrm{true}\end{array}}\,{\supset}\mathrm{I}^{u} \qquad \frac{\begin{array}{c}\\ u : A\end{array}^{\,u}}{\begin{array}{c}\vdots\\ M : B\end{array}}\,{\supset}\mathrm{I}^{u}$$

$$A \supset B\,\mathrm{true},\qquad \lambda u.M : A \supset B.$$

- Elimination rule

$$\frac{A \supset B\,\mathrm{true} \qquad A\,\mathrm{true}}{B\,\mathrm{true},}\,{\supset}\mathrm{E} \qquad \frac{M : A \supset B \qquad N : A}{M\,N : B.}\,{\supset}\mathrm{E}$$

- Reduction rule

$$(\lambda u.M)\,N \Rightarrow_R [\,N/u\,]\,M.$$

Typed Lambda Calculus (or Proofs as Programs)

Example

(i) A proof that $A \supset A\,\mathrm{true}$.

$$\dfrac{\dfrac{}{A\,\mathrm{true}}\,u}{A \supset A\,\mathrm{true}}\,\supset\mathrm{I}^u$$

(ii) A proof that $\lambda u.u : A \supset A$.

$$\dfrac{\dfrac{}{u : A}\,u}{\lambda u.u : A \supset A}\,\supset\mathrm{I}^u$$

Typed Lambda Calculus (or Proofs as Programs)

Example

(i) A proof that $(A \wedge B) \supset (B \wedge A)\,\mathrm{true}$.

$$\dfrac{\dfrac{\dfrac{\dfrac{}{A \wedge B\,\mathrm{true}}\,x}{B\,\mathrm{true}}\wedge\mathrm{E}_2 \quad \dfrac{\dfrac{}{A \wedge B\,\mathrm{true}}\,x}{A\,\mathrm{true}}\wedge\mathrm{E}_1}{B \wedge A\,\mathrm{true}}\wedge\mathrm{I}}{(A \wedge B) \supset (B \wedge A)\,\mathrm{true}}\supset\mathrm{I}^x$$

(ii) A proof that $\lambda x.\,\langle\,\mathsf{snd}\,x\,,\,\mathsf{fst}\,x\,\rangle : (A \wedge B) \supset (B \wedge A)$.

$$\dfrac{\dfrac{\dfrac{\dfrac{}{x : A \wedge B}\,x}{\mathsf{snd}\,x : B}\wedge\mathrm{E}_2 \quad \dfrac{\dfrac{}{x : A \wedge B}\,x}{\mathsf{fst}\,x : A}\wedge\mathrm{E}_1}{\langle\,\mathsf{snd}\,x\,,\,\mathsf{fst}\,x\,\rangle : B \wedge A}\wedge\mathrm{I}}{\lambda x.\,\langle\,\mathsf{snd}\,x\,,\,\mathsf{fst}\,x\,\rangle : (A \wedge B) \supset (B \wedge A)}\supset\mathrm{I}^x$$

Typed Lambda Calculus (or Proofs as Programs)

Example

(i) A proof that $A \supset (B \supset A)\,\mathrm{true}$.

$$\dfrac{\dfrac{\dfrac{}{A\,\mathrm{true}}\,u}{B \supset A\,\mathrm{true}}\,\supset\mathrm{I}}{A \supset (B \supset A)\,\mathrm{true}}\,\supset\mathrm{I}^{u}$$

(ii) A proof that $\lambda u.\lambda w.u : A \supset (B \supset A)$.

$$\dfrac{\dfrac{\dfrac{}{u : A}\,u}{\lambda w.u : B \supset A}\,\supset\mathrm{I}}{\lambda u.\lambda w.u : A \supset (B \supset A)}\,\supset\mathrm{I}^{u}$$

Typed Lambda Calculus (or Proofs as Programs)

Notation

Let Γ be a set of judgements and let $M : A$ be a judgement. The relation $\Gamma \vdash M : A$ means that there is a derivation with conclusion $M : A$ from the set of hypotheses Γ.

Typed Lambda Calculus (or Proofs as Programs)

Example

(i) A proof that $A \wedge B\,\mathrm{true} \vdash B \wedge A\,\mathrm{true}$.

$$\dfrac{\dfrac{A \wedge B\,\mathrm{true}}{B\,\mathrm{true}}\wedge\mathrm{E}_2 \qquad \dfrac{A \wedge B\,\mathrm{true}}{A\,\mathrm{true}}\wedge\mathrm{E}_1}{B \wedge A\,\mathrm{true}}\wedge\mathrm{I}$$

(ii) A proof that $M : A \wedge B \vdash \langle\, \mathsf{snd}\, M\,,\, \mathsf{fst}\, M \,\rangle : B \wedge A$.

$$\dfrac{\dfrac{M : A \wedge B}{\mathsf{snd}\, M : B}\wedge\mathrm{E}_2 \qquad \dfrac{M : A \wedge B}{\mathsf{fst}\, M : A}\wedge\mathrm{E}_1}{\langle\, \mathsf{snd}\, M\,,\, \mathsf{fst}\, M \,\rangle : B \wedge A}\wedge\mathrm{I}$$

Typed Lambda Calculus (or Proofs as Programs)

Inference rules for disjunction (sum type or coproduct type)

Typed Lambda Calculus (or Proofs as Programs)

Inference rules for disjunction (sum type or coproduct type)

- Formation rule

$$\frac{A\,\mathrm{prop} \qquad B\,\mathrm{prop}}{A \vee B\,\mathrm{prop},} \qquad \frac{A\,\mathrm{type} \qquad B\,\mathrm{type}}{A \vee B\,\mathrm{type}.}$$

(continued on next slide)

Typed Lambda Calculus (or Proofs as Programs)

Inference rules for disjunction (sum type or coproduct type) (continuation)

- Introduction rules

$$\frac{A\,\mathrm{true}}{A \vee B\,\mathrm{true},}\ \vee\mathrm{I}_1 \qquad \frac{M : A}{\mathsf{inl}\, M : A \vee B.}\ \vee\mathrm{I}_1$$

$$\frac{B\,\mathrm{true}}{A \vee B\,\mathrm{true},}\ \vee\mathrm{I}_2 \qquad \frac{N : B}{\mathsf{inr}\, N : A \vee B.}\ \vee\mathrm{I}_2$$

Typed Lambda Calculus (or Proofs as Programs)

Inference rules for disjunction (sum type or coproduct type) (continuation)

- Introduction rules

$$\frac{A\,\mathrm{true}}{A \vee B\,\mathrm{true},}\ \vee\mathrm{I}_1 \qquad \frac{M : A}{\mathsf{inl}\,M : A \vee B.}\ \vee\mathrm{I}_1$$

$$\frac{B\,\mathrm{true}}{A \vee B\,\mathrm{true},}\ \vee\mathrm{I}_2 \qquad \frac{N : B}{\mathsf{inr}\,N : A \vee B.}\ \vee\mathrm{I}_2$$

- Elimination rule

$$\frac{\begin{array}{ccc}\overset{u}{\overline{A\,\mathrm{true}}} & & \overset{w}{\overline{B\,\mathrm{true}}} \\ \vdots & & \vdots \\ A \vee B\,\mathrm{true} & C\,\mathrm{true} & C\,\mathrm{true}\end{array}}{C\,\mathrm{true},}\ \vee\mathrm{E}^{u,w} \qquad \frac{\begin{array}{ccc}\overset{u}{\overline{u : A}} & & \overset{w}{\overline{w : B}} \\ \vdots & & \vdots \\ M : A \vee B & N : C & P : C\end{array}}{\mathsf{case}(M, u.N, w.P) : C.}\ \vee\mathrm{E}^{u,w}$$

Typed Lambda Calculus (or Proofs as Programs)

Inference rules for disjunction (sum type or coproduct type) (continuation)

- Introduction rules

$$\frac{A\,\mathrm{true}}{A \vee B\,\mathrm{true},}\ \vee\mathrm{I}_1 \qquad \frac{M : A}{\mathsf{inl}\,M : A \vee B.}\ \vee\mathrm{I}_1$$

$$\frac{B\,\mathrm{true}}{A \vee B\,\mathrm{true},}\ \vee\mathrm{I}_2 \qquad \frac{N : B}{\mathsf{inr}\,N : A \vee B.}\ \vee\mathrm{I}_2$$

- Elimination rule

$$\frac{\begin{array}{ccc}\overset{u}{\overline{A\,\mathrm{true}}} & & \overset{w}{\overline{B\,\mathrm{true}}} \\ \vdots & & \vdots \\ A \vee B\,\mathrm{true} & C\,\mathrm{true} & C\,\mathrm{true}\end{array}}{C\,\mathrm{true},}\ \vee\mathrm{E}^{u,w} \qquad \frac{\begin{array}{ccc}\overset{u}{\overline{u : A}} & & \overset{w}{\overline{w : B}} \\ \vdots & & \vdots \\ M : A \vee B & N : C & P : C\end{array}}{\mathsf{case}(M, u.N, w.P) : C.}\ \vee\mathrm{E}^{u,w}$$

- Reduction rules

$$\mathsf{case}(\mathsf{inl}\,M, u.N, w.P) \Rightarrow_R [\,M/u\,]\,N,$$
$$\mathsf{case}(\mathsf{inr}\,M, u.N, w.P) \Rightarrow_R [\,M/u\,]\,P.$$

Typed Lambda Calculus (or Proofs as Programs)

Example

(i) A proof that $(A \vee B) \supset (B \vee A)\,\mathrm{true}$.

$$\dfrac{\dfrac{}{A \vee B\,\mathrm{true}}\,u \qquad \dfrac{\dfrac{}{A\,\mathrm{true}}\,v}{B \vee A\,\mathrm{true}}\vee\mathrm{I}_2 \qquad \dfrac{\dfrac{}{B\,\mathrm{true}}\,w}{B \vee A\,\mathrm{true}}\vee\mathrm{I}_1}{\dfrac{B \vee A\,\mathrm{true}}{(A \vee B) \supset (B \vee A)\,\mathrm{true}}\supset\mathrm{I}^u}\vee\mathrm{E}^{v,w}$$

(ii) A proof that $\lambda u.\,\mathsf{case}(u, v.\,\mathsf{inr}\,v, w.\,\mathsf{inl}\,w) : (A \vee B) \supset (B \vee A)$.

$$\dfrac{\dfrac{\dfrac{}{u : A \vee B}\,u \qquad \dfrac{\dfrac{}{v : A}\,v}{\mathsf{inr}\,v : B \vee A}\vee\mathrm{I}_2 \qquad \dfrac{\dfrac{}{w : B}\,w}{\mathsf{inl}\,w : B \vee A}\vee\mathrm{I}_1}{\mathsf{case}(u, v.\mathsf{inr}\,v, w.\mathsf{inl}\,w) : B \vee A}\vee\mathrm{E}^{v,w}}{\lambda u.\,\mathsf{case}(u, v.\,\mathsf{inr}\,v, w.\,\mathsf{inl}\,w) : (A \vee B) \supset (B \vee A)}\supset\mathrm{I}^u$$

Typed Lambda Calculus (or Proofs as Programs)

Inference rules for truth (unit type)

Typed Lambda Calculus (or Proofs as Programs)

Inference rules for truth (unit type)

- Formation rule

$$\frac{}{\top\ \mathrm{prop},} \qquad \frac{}{\top\ \mathrm{type}.}$$

Typed Lambda Calculus (or Proofs as Programs)

Inference rules for truth (unit type)

- Formation rule

$$\frac{}{\top\ \mathrm{prop},} \qquad \frac{}{\top\ \mathrm{type}.}$$

- Introduction rule

$$\frac{}{\top\ \mathrm{true},}\ \top\mathrm{I} \qquad \frac{}{\langle\rangle : \top.}\ \top\mathrm{I}$$

Typed Lambda Calculus (or Proofs as Programs)

Inference rules for truth (unit type)

- Formation rule

$$\frac{}{\top\ \mathrm{prop},} \qquad \frac{}{\top\ \mathrm{type}.}$$

- Introduction rule

$$\frac{}{\top\ \mathrm{true},}\ \top\mathrm{I} \qquad \frac{}{\langle\rangle : \top.}\ \top\mathrm{I}$$

- No elimination rule
- No reduction rule

Typed Lambda Calculus (or Proofs as Programs)

Inference rules for falsehood (empty type)

Typed Lambda Calculus (or Proofs as Programs)

Inference rules for falsehood (empty type)

- Formation rule

$$\frac{}{\bot\,\mathrm{prop},} \qquad \frac{}{\bot\,\mathrm{type}.}$$

Typed Lambda Calculus (or Proofs as Programs)

Inference rules for falsehood (empty type)

- Formation rule

$$\frac{}{\bot\,\mathrm{prop},} \qquad \frac{}{\bot\,\mathrm{type}.}$$

- Elimination rule

$$\frac{\bot\,\mathrm{true}}{A\,\mathrm{true},}\,\bot\mathrm{E} \qquad \frac{M : \bot}{\mathsf{abort}\,M : A.}\,\bot\mathrm{E}$$

Typed Lambda Calculus (or Proofs as Programs)

Inference rules for falsehood (empty type)

- Formation rule

$$\frac{}{\bot\ \mathrm{prop},} \qquad \frac{}{\bot\ \mathrm{type}.}$$

- Elimination rule

$$\frac{\bot\ \mathrm{true}}{A\ \mathrm{true},}\ \bot\mathrm{E} \qquad \frac{M : \bot}{\mathsf{abort}\ M : A.}\ \bot\mathrm{E}$$

- No introduction rule
- No reduction rule

Typed Lambda Calculus (or Proofs as Programs)

Guiding principles behind the proposition-as-types principle

Textbook (p. L4.6):

(i) *Every proposition corresponds to a type and vice versa.*
(ii) *Introduction rules correspond to value constructors.*
(iii) *Elimination rules correspond to value destructors.*
(iv) *For every deduction of* $A\,\mathrm{true}$ *there is a proof term* M *and deduction of* $M : A$*. We can effectively construct this top-down.*
(v) *For every deduction of* $M : A$ *there is a deduction of* $A\,\mathrm{true}$*. We can effective construct this bottom-up.*

Typed Lambda Calculus (or Proofs as Programs)

Agda implementation of product types (conjunction)

- Formation and introduction rules

$$\frac{A\,\text{type} \qquad B\,\text{type}}{A \times B\,\text{type},}$$

$$\frac{M : A \qquad N : B}{\langle M\,,\,N \rangle : A \times B.}\ \times\text{I}$$

```
1  data _×_ (A B : Set) : Set where
2    ⟨_,_⟩ : A → B → A × B
```

(continued on next slide)

Typed Lambda Calculus (or Proofs as Programs)

Agda implementation of product types (conjunction) (continuation)

- Elimination and reduction rules

$$\frac{M : A \times B}{\mathsf{fst}\, M : A,} \times\mathrm{E}_1$$

$$\frac{M : A \times B}{\mathsf{snd}\, M : B,} \times\mathrm{E}_2$$

$$\mathsf{fst} \langle M , N \rangle \Rightarrow_R M,$$
$$\mathsf{snd} \langle M , N \rangle \Rightarrow_R N.$$

```
1  fst : {A B : Set} → A × B → A
2  fst ⟨ a , _ ⟩ = a
3
4  snd : {A B : Set} → A × B → B
5  snd ⟨ _ , b ⟩ = b
```

Typed Lambda Calculus (or Proofs as Programs)

Agda implementation of function types (implication)

The function type $\rightarrow$ is built-in in Agda.

Typed Lambda Calculus (or Proofs as Programs)

Example

A proof and an implementation that $\lambda a.a : A \to A$.

$$\dfrac{\dfrac{}{a : A}\,a}{\lambda a.a : A \to A}\,{\to}\mathrm{I}^{a}$$

```
1   id₁ : {A : Set} → A → A
2   id₁ = λ a → a
3
4   id₂ : {A : Set} → A → A
5   id₂ a = a
```

Typed Lambda Calculus (or Proofs as Programs)

Example

A proof and an implementation that $\lambda x.\, \langle\, \mathsf{snd}\, x\,,\, \mathsf{fst}\, x\,\rangle : (A \times B) \to (B \times A)$.

$$\dfrac{\dfrac{\dfrac{\dfrac{}{x : A \times B}\, x}{\mathsf{snd}\, x : B}\times\mathrm{E}_2 \qquad \dfrac{\dfrac{}{x : A \times B}\, x}{\mathsf{fst}\, x : A}\times\mathrm{E}_1}{\langle\, \mathsf{snd}\, x\,,\, \mathsf{fst}\, x\,\rangle : B \times A}\times\mathrm{I}}{\lambda x.\, \langle\, \mathsf{snd}\, x\,,\, \mathsf{fst}\, x\,\rangle : (A \times B) \to (B \times A)}\to\mathrm{I}^x$$

```
1  ×-comm₁ : {A B : Set} → A × B → B × A
2  ×-comm₁ = λ x → ⟨ snd x ,  fst x ⟩
3
4  -- Using pattern-matching
5  ×-comm₂ : {A B : Set} → A × B → B × A
6  ×-comm₂ ⟨ a , b ⟩ = ⟨ b ,  a ⟩
```

Typed Lambda Calculus (or Proofs as Programs)

Example

A proof and an implementation that $\lambda a.\lambda b.a : A \to (B \to A)$.

$$\dfrac{\dfrac{\dfrac{}{a : A}\,a}{\lambda b.a : B \to A}\,{\to}\mathrm{I}}{\lambda a.\lambda b.a : A \to (B \to A)}\,{\to}\mathrm{I}^{a}$$

```
1  ax₁ : {A B : Set} → A → (B → A)
2  ax₁ = λ a _ → a
3
4  ax₂ : {A B : Set} → A → (B → A)
5  ax₂ a _ = a
```

Typed Lambda Calculus (or Proofs as Programs)

Agda implementation of sum types (disjunction)

- Formation and introduction rules

$$\frac{A\,\text{type} \qquad B\,\text{type}}{A + B\,\text{type},}$$

$$\frac{M : A}{\mathsf{inl}\, M : A + B,}\ +\mathrm{I}_1$$

$$\frac{N : B}{\mathsf{inr}\, N : A + B.}\ +\mathrm{I}_2$$

```
1  data _+_ (A B : Set) : Set where
2    inl : A → A + B
3    inr : B → A + B
```

(continued on next slide)

Typed Lambda Calculus (or Proofs as Programs)

Agda implementation of sum types (disjunction) (continuation)

- Elimination and reduction rules

$$\dfrac{M : A + B \qquad \begin{array}{c}\overline{u : A}^{\,u} \\ \vdots \\ N : C\end{array} \qquad \begin{array}{c}\overline{w : B}^{\,w} \\ \vdots \\ P : C\end{array}}{\mathsf{case}(M, u.N, w.P) : C,}\ +\mathrm{E}^{u,w}$$

$$\mathsf{case}(\mathsf{inl}\, M, u.N, w.P) \Rightarrow_R [\, M/u \,]\, N,$$
$$\mathsf{case}(\mathsf{inr}\, M, u.N, w.P) \Rightarrow_R [\, M/u \,]\, P.$$

```
1  case : {A B C : Set} →
2         A + B →
3         (A → C) →
4         (B → C) →
5         C
6  case (inl a) f g = f a
7  case (inr b) f g = g b
```

Typed Lambda Calculus (or Proofs as Programs)

Example

A proof and an implementation that $\lambda u.\,\mathsf{case}(x, a.\,\mathsf{inr}\,a, b.\,\mathsf{inl}\,b) : (A + B) \to (B + A)$.

$$\dfrac{\dfrac{\dfrac{}{u : A + B}\,x \qquad \dfrac{\dfrac{}{a : A}\,a}{\mathsf{inr}\,v : B + A}\,{+}\mathrm{I}_2 \qquad \dfrac{\dfrac{}{b : B}\,b}{\mathsf{inl}\,b : B + A}\,{+}\mathrm{I}_1}{\mathsf{case}(x, a.\mathsf{inr}\,a, b.\mathsf{inl}\,b) : B + A}\,{+}\mathrm{E}^{a,b}}{\lambda x.\,\mathsf{case}(x, a.\,\mathsf{inr}\,a, b.\,\mathsf{inl}\,b) : (A + B) \to (B + A)}\,{\to}\mathrm{I}^{x}$$

```
1  +-comm₁ : {A B : Set} → A + B → B + A
2  +-comm₁ = λ x → case x (λ a → inr a) (λ b → inl b)
3
4  -- Using pattern-matching
5  +-comm₂ : {A B : Set} → A + B → B + A
6  +-comm₂ (inl a) = inr a
7  +-comm₂ (inr b) = inl b
```

Typed Lambda Calculus (or Proofs as Programs)

Agda implementation of unit type (truth)

- Formation and introduction rules

$$\frac{}{\top \text{ type},}$$

$$\frac{}{\langle\rangle : \top.}\ \top\mathrm{I}$$

```
1  data ⊤ : Set where
2    ⟨⟩ : ⊤
```

Typed Lambda Calculus (or Proofs as Programs)

Agda implementation of empty type (falsehood)

- Formation rule

$$\frac{}{\bot \text{ type.}}$$

```
data ⊥ : Set where
```

Typed Lambda Calculus (or Proofs as Programs)

Agda implementation of empty type (falsehood)

- Formation rule

$$\frac{}{\bot\,\mathrm{type}.}$$

```
data ⊥ : Set where
```

- Elimination rule

$$\frac{M : \bot}{\mathsf{abort}\, M : A.}\ \bot\mathrm{E}$$

```
1  ⊥-elim : {A : Set} → ⊥ → A
2  ⊥-elim ()
```

References


Frank Pfenning (26th Jan. 2023). Lecture Notes on Proofs as Programs. Material for the course 15-317 Constructive Logic at Carnegie Mellon University, Spring 2023. URL: https://www.cs.cmu.edu/~fp/courses/15317-s23/ (cit. on p. 2).


Morten-Heine Sørensen and Paul Urzyczyn (2006). Lectures on the Curry-Howard Isomorphism. Vol. 149. Studies in Logic and the Foundations of Mathematics. Elsevier (cit. on p. 12).